NEVYL DIGITAL BANKING

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Name: Nevyl Digital Banking

Document Title: Software Requirements Specification

Document ID: NEVYL-SRS-001

Version: 1.0

Status: Draft — Baseline for Development

Prepared By: Naftal Bosire

Date: 9 November 2026

Project Type: Digital Banking Simulation

Document Classification: Project Documentation

DOCUMENT CONTROL

Revision History

Version	Date	Description	Author
1.0	09 November 2026	Initial SRS baseline	Naftal Bosire

Approval

Role	Name/Organization	Approval
Product Owner	Naftal Bosire	
Software Developer	Naftal Bosire	

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the Nevyl Digital Banking system.

The document establishes the scope, behavior, constraints, interfaces, security requirements, data requirements, and expected performance of the system.

It serves as a reference for:

- Software development.
- System architecture and database design.
- Software testing and quality assurance.
- Project planning and implementation.
- Technical documentation.
- Future system maintenance and enhancement.

The requirements defined in this document shall guide the implementation of Nevyl's Minimum Viable Product (MVP).

1.2 Scope

Nevyl Digital Banking is a multi-currency digital banking simulation consisting of an Android mobile application and a backend banking system.

The system will allow customers to register, create bank accounts, manage simulated funds, transfer funds between eligible accounts, and view their transaction history.

The backend will implement a double-entry ledger to maintain financial integrity. It will provide authentication, authorization, transaction processing, account management, audit logging, and administrative functionality.

Nevyl is intended for educational, demonstration, and portfolio purposes. It will not initially process real money or connect to actual banking or payment networks.

1.2.1 In-Scope Features

The MVP shall include:

- Customer registration and authentication.
- Staff-created customer registration.
- User profile management.
- Multi-currency bank account management.

- Simulated deposits and withdrawals.
- Internal account-to-account transfers.
- Double-entry ledger accounting.
- Transaction history and receipts.
- Account freezing and unfreezing.
- Role-based access control.
- Staff operational functionality.
- Transaction reversals.
- Audit logging.
- Idempotency protection.
- Concurrent transaction protection.
- Automated testing.
- API documentation.
- Containerized deployment.

1.2.2 Out-of-Scope Features

The following shall not be included in the initial MVP:

- Real monetary transactions.
- Live M-Pesa integration.
- Real bank-to-bank transfers.
- Actual payment gateway integration.
- Foreign exchange conversion.
- Loan origination and credit scoring.
- Debit or credit card issuance.
- Regulatory KYC verification.
- Biometric authentication.
- Real-time interbank settlement.
- Production banking infrastructure.

These features may be considered in future releases.

1.3 Intended Audience

This document is intended for:

- Product owners.
- Software developers.
- System architects.
- Database designers.
- Quality assurance engineers.
- Technical reviewers.
- Fintech recruiters evaluating the project.

1.4 Definitions and Acronyms

TERM	DEFINITION
SRS	Software Requirements Specification
MVP	Minimum Viable Product
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
RBAC	Role-Based Access Control
DBMS	Database Management System
ACID	Atomicity, Consistency, Isolation, Durability
KES	Kenyan Shilling
USD	United States Dollar
EUR	Euro
GBP	British Pound Sterling
LEDGER	A record of financial transactions and their debit and credit entries
IDEMPOTENCY	Property that prevents repeated requests from executing the same operation more than once
REVERSAL	A compensating transaction used to counteract a previously posted transaction
CUSTOMER ACCOUNT	A bank account associated with a customer
INTERNAL ACCOUNT	A bank-controlled ledger account used to represent simulated funding, settlement, or other accounting balances

1.5 References

The following references shall guide the project:

- ISO/IEC/IEEE 29148 — Requirements Engineering.
- ISO/IEC 25010 — Systems and Software Quality Models.
- OWASP Application Security Verification Standard (ASVS).
- OWASP API Security Top 10.
- Java 21 documentation.
- Spring Boot documentation.
- PostgreSQL documentation.
- Android Developers documentation.

2. OVERALL DESCRIPTION

2.1 Product Perspective

Nevyl shall be developed as a client-server application.

The Android application shall communicate with the backend through secure REST APIs. The backend shall process requests, enforce business rules, manage financial transactions, and persist information in a PostgreSQL database.

The system shall initially use a modular monolithic architecture.

The backend shall be designed to support future clients, such as a web-based administrative dashboard, without requiring the core banking logic to be rewritten.

2.1.1 High-Level System Architecture

The system shall consist of the following components:

1. Android Mobile Application.
2. REST API Backend.
3. Authentication and Authorization Module.
4. Customer and Account Management Module.
5. Transaction Processing Module.
6. Ledger Management Module.
7. Audit and Reporting Module.
8. PostgreSQL Database.

2.2 Product Functions

At a high level, Nevyl shall:

- Authenticate customers and staff.
- Manage customer profiles.
- Create and manage bank accounts.
- Maintain account balances through ledger transactions.
- Process simulated deposits and withdrawals.
- Process internal account transfers.
- Maintain transaction histories.
- Enforce account ownership and access permissions.
- Record audit events.

- Support transaction reversals.
- Provide administrative operations.

2.3 User Classes and Characteristics

2.3.1 Customer

Customers are registered users who access the banking application.

Customers shall be able to:

- Register and authenticate.
- View their profile.
- Create eligible bank accounts.
- View their own account balances.
- Deposit and withdraw simulated funds.
- Transfer funds between eligible accounts.
- View their transaction history.
- View transaction receipts.
- Freeze and unfreeze their own accounts.

Customers shall not have access to other customers' private account information.

2.3.2 Teller

Tellers are authorized bank employees who perform operational transactions.

Tellers shall be able to:

- Authenticate using staff credentials.
- Search for authorized customer accounts.
- Process simulated deposits.
- Process simulated withdrawals.
- View permitted transaction details.

Tellers shall not manage system roles or configuration.

2.3.3 Manager

Managers supervise banking operations.

Managers shall be able to:

- Review transactions.
- Review operational activity.

- Approve selected operations where approval workflows apply.
- Initiate transaction reversals.
- View audit records within their permitted scope.

2.3.4 Administrator

Administrators manage system-level functionality.

Administrators shall be able to:

- Manage user accounts.
- Assign and revoke roles.
- Manage account configurations.
- Configure supported currencies.
- Access system audit records.
- Manage selected system settings.

Administrative privileges shall not automatically permit unauthorized alteration of posted financial records.

2.4 Operating Environment

2.4.1 Android Application

The mobile application shall operate on compatible Android devices.

The initial development environment shall use:

- Kotlin.
- Jetpack Compose.
- Android SDK.
- MVVM architecture.
- Retrofit for API communication.

The minimum supported Android version shall be established during implementation.

2.4.2 Backend

The backend shall use:

- Java 21.
- Spring Boot.
- Spring Security.
- Spring Data JPA.
- Hibernate.

- REST APIs.
- JWT-based authentication.

2.4.3 Database

The database shall use PostgreSQL.

Flyway shall manage database schema migrations.

2.4.4 Infrastructure

The system shall support:

- Docker-based local development.
- Git version control.
- GitHub repository management.
- Automated testing.
- CI/CD workflows.

2.5 Design and Implementation Constraints

- The backend shall be implemented using Java and Spring Boot.
- The database shall be PostgreSQL.
- The mobile application shall be Android-based.
- Financial amounts shall not use floating-point types.
- Posted ledger entries shall not be directly edited or deleted.
- Financial transactions shall be atomic.
- Account balances shall be derived from or consistently maintained alongside ledger records.
- Access control shall be enforced on the backend.
- The initial application shall simulate all financial activity.

2.6 User Documentation

The project shall provide:

- A project README.
- API documentation.
- A developer setup guide.
- A system design document.
- A database design document.
- A user manual upon completion of the MVP.

3. SYSTEM FEATURES AND FUNCTIONAL REQUIREMENTS

Each requirement shall have a unique identifier and shall be testable.

3.1 User Registration and Authentication

FR-AUTH-001: Customer Registration

The system shall allow customers to register using required personal information and valid authentication credentials.

FR-AUTH-002: Registration Validation

The system shall validate submitted registration data and reject invalid or incomplete information.

FR-AUTH-003: Unique User Identity

The system shall enforce uniqueness for designated user identity fields, including email address and username where applicable.

FR-AUTH-004: Customer Login

The system shall allow registered users to authenticate using valid credentials.

FR-AUTH-005: Password Security

The system shall store passwords using a secure, one-way password hashing algorithm.

FR-AUTH-006: Authentication Tokens

The system shall issue authentication tokens to successfully authenticated users.

FR-AUTH-007: Unauthorized Access

The system shall reject requests to protected resources when valid authentication is not provided.

FR-AUTH-008: Token Expiration

The system shall enforce authentication token expiration.

FR-AUTH-009: Staff-Created Customers

The system shall allow authorized staff members to create customer records.

FR-AUTH-010: User Account Status

The system shall support user account statuses such as ACTIVE, SUSPENDED, and DISABLED.

FR-AUTH-011: Account Suspension

The system shall prevent suspended or disabled users from performing unauthorized authenticated operations.

3.2 Customer Profile Management

FR-CUS-001: View Profile

The system shall allow authenticated customers to view their own profile information.

FR-CUS-002: Update Profile

The system shall allow customers to update permitted profile fields.

FR-CUS-003: Profile Ownership

The system shall prevent customers from accessing or modifying another customer's profile without appropriate authorization.

FR-CUS-004: Staff Profile Access

The system shall allow authorized staff to view customer information according to their assigned permissions.

3.3 Bank Account Management

FR-ACC-001: Create Account

The system shall allow eligible customers to create a simulated bank account.

FR-ACC-002: Account Number Generation

The system shall generate a unique account number for each bank account.

FR-ACC-003: Currency Assignment

Each bank account shall be assigned exactly one supported currency at creation.

FR-ACC-004: Account Details

The system shall allow customers to view their account number, account type, currency, status, and balance.

FR-ACC-005: Account Ownership

The system shall enforce account ownership when customers access account resources.

FR-ACC-006: Account Status

The system shall support account statuses including ACTIVE, FROZEN, and CLOSED.

FR-ACC-007: Account Freezing

The system shall allow customers to freeze their own eligible accounts.

FR-ACC-008: Account Unfreezing

The system shall allow customers to unfreeze their own eligible accounts unless a staff-imposed restriction applies.

FR-ACC-009: Frozen Account Restrictions

The system shall prevent prohibited financial operations involving frozen accounts.

FR-ACC-010: Account Closure

The system shall support account closure subject to defined business rules.

3.4 Simulated Deposits

FR-DEP-001: Deposit Initiation

The system shall allow authorized customers or staff to initiate simulated deposits.

FR-DEP-002: Deposit Amount Validation

The system shall reject deposits with zero, negative, or invalid monetary amounts.

FR-DEP-003: Deposit Posting

The system shall record a successful deposit as a balanced ledger transaction.

FR-DEP-004: Deposit Reference

The system shall assign a unique transaction reference to each successfully posted deposit.

FR-DEP-005: Deposit History

The system shall make successful deposits available in the relevant account transaction history.

FR-DEP-006: Deposit Currency

The deposit currency shall match the target account currency.

3.5 Simulated Withdrawals

FR-WDR-001: Withdrawal Initiation

The system shall allow authorized customers or staff to initiate simulated withdrawals.

FR-WDR-002: Withdrawal Amount Validation

The system shall reject zero, negative, or invalid withdrawal amounts.

FR-WDR-003: Sufficient Funds

The system shall reject withdrawals that exceed the available balance.

FR-WDR-004: Withdrawal Posting

The system shall record a successful withdrawal as a balanced ledger transaction.

FR-WDR-005: Withdrawal Reference

The system shall assign a unique transaction reference to each successfully posted withdrawal.

FR-WDR-006: Withdrawal Currency

The withdrawal currency shall match the account currency.

3.6 Internal Transfers

FR-TRF-001: Transfer Initiation

The system shall allow customers to transfer funds between eligible Nevyl accounts.

FR-TRF-002: Recipient Validation

The system shall validate that the destination account exists and is eligible to receive funds.

FR-TRF-003: Sender Validation

The system shall validate that the source account is active and owned by the initiating customer.

FR-TRF-004: Currency Compatibility

The system shall reject transfers between accounts with different currencies.

FR-TRF-005: Sufficient Funds

The system shall verify that the source account has sufficient available funds.

FR-TRF-006: Atomic Transfer

The system shall debit the source account and credit the destination account within one atomic database transaction.

FR-TRF-007: Balanced Ledger

The system shall create balanced debit and credit ledger entries for each successful transfer.

FR-TRF-008: Transfer Reference

The system shall assign a unique transaction reference to each successful transfer.

FR-TRF-009: Transfer History

The system shall record transfer details in the relevant account transaction histories.

FR-TRF-010: Duplicate Prevention

The system shall prevent duplicate transfer execution caused by repeated requests.

FR-TRF-011: Concurrent Transfers

The system shall protect account balances against concurrent transaction conflicts.

3.7 Transaction History and Receipts

FR-HIS-001: Transaction History

The system shall allow customers to view transactions associated with their own accounts.

FR-HIS-002: Transaction Details

The system shall provide transaction details including:

- Transaction reference.
- Transaction type.
- Amount.
- Currency.
- Status.
- Timestamp.

- Description.
- Relevant account details.

FR-HIS-003: History Filtering

The system shall support filtering transaction history by transaction type, status, and date range.

FR-HIS-004: History Pagination

The system shall support pagination for transaction history retrieval.

FR-HIS-005: Receipt Generation

The system shall provide a receipt representation for successful transactions.

FR-HIS-006: Transaction Status

The system shall distinguish between transaction statuses such as PENDING, POSTED, FAILED, and REVERSED where applicable.

3.8 Ledger Management

FR-LED-001: Double-Entry Accounting

The system shall implement double-entry bookkeeping for posted financial transactions.

FR-LED-002: Balanced Transactions

The total debit amount shall equal the total credit amount for each posted transaction within a single currency.

FR-LED-003: Ledger Entries

Each posted transaction shall contain the required ledger entries.

FR-LED-004: Immutable Ledger

The system shall prohibit direct modification or deletion of posted ledger entries.

FR-LED-005: Internal Ledger Accounts

The system shall maintain internal ledger accounts for simulated funding, cash settlement, and other required accounting operations.

FR-LED-006: Account Balance Consistency

The system shall maintain account balance consistency with the underlying ledger.

FR-LED-007: Ledger Transaction Reference

Each ledger transaction shall have a unique identifier or reference.

FR-LED-008: Transaction Atomicity

The system shall ensure that incomplete financial transactions are not committed.

FR-LED-009: Currency Separation

The system shall not combine ledger amounts belonging to different currencies when validating transaction balance.

3.9 Transaction Reversals

FR-REV-001: Reversal Initiation

The system shall allow authorized managers or administrators to initiate transaction reversals.

FR-REV-002: Reversal Eligibility

The system shall validate whether the original transaction is eligible for reversal.

FR-REV-003: Compensating Entries

The system shall create compensating ledger entries for a reversal.

FR-REV-004: Original Transaction Preservation

The system shall preserve the original transaction and its ledger entries.

FR-REV-005: Reversal Reference

The system shall record a unique reversal reference linked to the original transaction.

FR-REV-006: Duplicate Reversal Prevention

The system shall prevent unauthorized or duplicate reversal of the same transaction.

FR-REV-007: Reversal Audit

The system shall record the initiating staff member, reason, timestamp, and affected transaction.

3.10 Role-Based Access Control

FR-RBAC-001: Role Assignment

The system shall associate users with one or more authorized roles.

FR-RBAC-002: Permission Enforcement

The system shall enforce permissions on protected backend operations.

FR-RBAC-003: Customer Restrictions

Customers shall only access resources permitted by their ownership and assigned permissions.

FR-RBAC-004: Staff Restrictions

Staff members shall only perform operations permitted by their assigned roles.

FR-RBAC-005: Role Administration

The system shall allow authorized administrators to assign and revoke roles.

FR-RBAC-006: Privilege Escalation Prevention

The system shall prevent users from granting themselves unauthorized roles or permissions.

3.11 Audit Logging

FR-AUD-001: Audit Event Recording

The system shall record security-sensitive and operational events.

FR-AUD-002: Audit Information

Audit records shall include, where applicable:

- Actor identifier.
- Event type.
- Target resource.
- Timestamp.
- Operation result.
- Relevant metadata.

FR-AUD-003: Financial Audit

The system shall record relevant financial transaction events.

FR-AUD-004: Role Change Audit

The system shall record role assignment and revocation events.

FR-AUD-005: Audit Integrity

The system shall prevent unauthorized modification or deletion of audit records.

FR-AUD-006: Sensitive Data Exclusion

The system shall not store plaintext passwords, authentication secrets, or unnecessary sensitive information in audit logs.

3.12 Idempotency and Concurrency

FR-IDM-001: Idempotency Key

The system shall support idempotency keys for applicable financial transaction requests.

FR-IDM-002: Duplicate Request Handling

The system shall return the appropriate prior result when a request with the same idempotency key is repeated.

FR-IDM-003: Request Consistency

The system shall reject reuse of an idempotency key with a materially different request payload.

FR-CON-001: Concurrent Balance Protection

The system shall prevent concurrent financial operations from causing incorrect account balances.

FR-CON-002: Transaction Isolation

The system shall use appropriate database transaction isolation or locking mechanisms for financial operations.

FR-CON-003: Atomic Posting

The system shall ensure that financial posting and related balance updates succeed or fail together.

4. EXTERNAL INTERFACE REQUIREMENTS

4.1 Android User Interface

The Android application shall provide a user interface for:

- Registration.
- Login.
- Dashboard.
- Account listing.
- Account details.
- Deposits.
- Withdrawals.
- Transfers.
- Transaction history.
- Transaction details and receipts.
- Profile management.
- Account freezing and unfreezing.

The interface shall provide validation feedback and clear transaction status messages.

The application shall not expose privileged staff operations to unauthorized customers.

4.2 REST API Interface

The backend shall expose RESTful APIs over HTTP.

The API shall:

- Use JSON for request and response bodies.
- Use appropriate HTTP methods.
- Return meaningful HTTP status codes.
- Validate request payloads.
- Enforce authentication and authorization.
- Provide consistent error responses.
- Document endpoints using OpenAPI/Swagger.

4.3 Database Interface

The backend shall interact with PostgreSQL through an appropriate persistence layer.

Database operations shall use parameterized queries or ORM-generated queries to mitigate SQL injection risks.

The system shall maintain relational integrity through database constraints.

4.4 Communication Interface

- Client-server communication shall use HTTPS in deployed environments.
- Authentication tokens shall be transmitted securely.
- Sensitive credentials shall not be exposed in URLs.
- API requests shall be validated before processing.

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Security

NFR-SEC-001: Password Storage

Passwords shall be stored using a secure password hashing algorithm such as Argon2id or BCrypt.

NFR-SEC-002: Authentication

Protected API operations shall require valid authentication.

NFR-SEC-003: Authorization

Authorization shall be enforced at the backend for every protected resource and operation.

NFR-SEC-004: Transport Security

Deployed client-server communication shall use HTTPS.

NFR-SEC-005: Input Validation

The system shall validate user input to reduce injection and malformed request risks.

NFR-SEC-006: Secret Management

Credentials, private keys, and sensitive configuration values shall not be committed to the source repository.

NFR-SEC-007: Sensitive Logging

The system shall not log plaintext passwords, access tokens, or sensitive authentication secrets.

NFR-SEC-008: Brute-Force Protection

The system shall implement appropriate controls to mitigate repeated unauthorized authentication attempts.

5.2 Financial Integrity

NFR-FIN-001: Monetary Precision

Financial values shall use decimal-safe representations.

The database shall use an appropriate fixed-precision decimal type, such as NUMERIC(19,4), and Java shall use BigDecimal.

NFR-FIN-002: Ledger Balance

Every posted financial transaction shall satisfy the double-entry balance requirement.

NFR-FIN-003: Atomicity

Financial operations shall be processed atomically.

NFR-FIN-004: No Negative Available Balance

Customer accounts shall not be permitted to overdraw unless an explicitly authorized overdraft feature is introduced in a future release.

NFR-FIN-005: Ledger Immutability

Posted financial ledger entries shall be immutable.

NFR-FIN-006: Reconciliation

The system shall provide a means of validating that account balance projections agree with ledger records.

5.3 Performance

NFR-PER-001: API Response Time

Under the defined local demonstration environment and normal load, at least 95% of non-complex API requests should complete within 500 milliseconds, excluding external network latency.

NFR-PER-002: Transaction Processing

Under normal load, at least 95% of successful internal transfers should complete within 2 seconds, excluding client network latency.

NFR-PER-003: Pagination

Transaction history endpoints shall use pagination to avoid returning unbounded result sets.

NFR-PER-004: Database Efficiency

The database shall use appropriate indexes for frequently queried fields.

5.4 Reliability and Availability

NFR-REL-001: Transaction Consistency

The system shall maintain database consistency during successful and failed transaction processing.

NFR-REL-002: Failure Handling

The system shall handle unexpected failures without committing incomplete financial operations.

NFR-REL-003: Health Monitoring

The backend shall expose application health information.

NFR-REL-004: Data Backup

The project shall provide a documented database backup and restoration procedure.

5.5 Maintainability

NFR-MNT-001: Modular Design

The backend shall be organized into logical modules with clear responsibilities.

NFR-MNT-002: Code Quality

The implementation shall follow appropriate Java coding conventions and SOLID principles.

NFR-MNT-003: Database Migrations

Database schema changes shall be managed through versioned migrations.

NFR-MNT-004: Documentation

The project shall maintain technical documentation sufficient for a developer to understand and run the system.

5.6 Testability

NFR-TST-001: Unit Testing

Core business logic shall be covered by unit tests.

NFR-TST-002: Integration Testing

The backend shall include integration tests for database-backed operations.

NFR-TST-003: Financial Testing

Financial transaction tests shall verify ledger balance, account balance consistency, and insufficient-funds behavior.

NFR-TST-004: Security Testing

Tests shall verify role restrictions, account ownership, and authentication requirements.

NFR-TST-005: Concurrency Testing

The system shall include tests for simultaneous financial operations.

5.7 Usability and Accessibility

NFR-UX-001: Usability

The Android application shall provide clear navigation and understandable financial transaction flows.

NFR-UX-002: Feedback

The application shall display meaningful feedback for successful, failed, and pending operations.

NFR-UX-003: Accessibility

The application shall support appropriate text scaling, readable contrast, and accessible labels for interactive elements.

NFR-UX-004: Error Presentation

The application shall present user-friendly error messages without exposing internal system details.

6. DATA REQUIREMENTS

6.1 Data Storage

The system shall use PostgreSQL as its primary relational database.

The database shall store:

- User records.
- Customer profiles.
- Roles and permissions.
- Bank accounts.
- Account types.
- Supported currencies.
- Ledger transactions.
- Ledger entries.
- Transaction metadata.
- Audit logs.
- Idempotency records.
- Authentication token records where required.

6.2 Data Integrity

The database shall enforce:

- Primary key constraints.
- Foreign key constraints.
- Unique constraints.
- Appropriate not-null constraints.
- Valid monetary amount constraints.
- Valid account and transaction status constraints.
- Currency consistency rules where applicable.

6.3 Monetary Data

Financial amounts shall:

- Use fixed-precision decimal storage.
- Avoid floating-point representation.

- Be associated with a currency code.
- Be validated before posting.

6.4 Data Retention

Posted financial transactions and ledger records shall be retained throughout the MVP lifecycle.

The system shall not permit customers or unauthorized staff to permanently delete posted financial records.

6.5 Data Privacy

The system shall restrict access to customer data based on authorization and ownership.

Only information necessary for the intended operation shall be returned through APIs.

7. BUSINESS RULES

7.1 Account Rules

BR-001

Every bank account shall belong to one customer.

BR-002

Every bank account shall have a unique account number.

BR-003

Every bank account shall have exactly one currency.

BR-004

A frozen account shall not be allowed to perform prohibited debit or credit operations.

BR-005

A closed account shall not be allowed to perform new financial transactions.

7.2 Currency Rules

BR-006

Nevyl shall support multiple currencies.

BR-007

The initial planned currencies are KES, USD, EUR, and GBP. The final supported currency list shall be confirmed during system design.

BR-008

Transfers between accounts shall require matching currency codes.

BR-009

Foreign exchange conversion shall not be supported in the MVP.

BR-010

Each currency shall have appropriate monetary precision and display rules.

7.3 Ledger Rules

BR-011

Every posted transaction shall have at least one debit entry and one credit entry.

BR-012

The sum of debit entries shall equal the sum of credit entries within the same currency.

BR-013

Posted ledger entries shall not be modified or deleted.

BR-014

Corrections shall be performed through compensating transactions.

BR-015

Customer account balances shall be consistent with their posted ledger entries.

7.4 Transfer Rules

BR-016

A transfer shall require an eligible source account and destination account.

BR-017

A transfer shall not be permitted if the source account has insufficient available funds.

BR-018

A customer shall not transfer funds from an account they do not own.

BR-019

A transfer shall be committed atomically.

BR-020

A transfer shall not be executed more than once for the same accepted idempotency request.

7.5 Simulated Funding Rules

BR-021

All deposits and withdrawals shall represent simulated banking activity.

BR-022

Simulated deposits shall be backed by an appropriate internal ledger account.

BR-023

Simulated withdrawals shall be posted against an appropriate internal ledger account.

BR-024

Simulated funding shall not be represented as real money received or paid.

7.6 Reversal Rules

BR-025

A reversal shall reference the original transaction.

BR-026

A reversal shall create compensating ledger entries.

BR-027

The original transaction shall remain available for audit and history purposes.

BR-028

A transaction shall not be reversed more than once unless an explicitly designed partial-reversal mechanism permits it.

8. SYSTEM CONSTRAINTS

8.1 Technical Constraints

- Backend implementation shall use Java 21 and Spring Boot.
- Database implementation shall use PostgreSQL.
- Mobile implementation shall use Kotlin and Jetpack Compose.
- The backend shall expose REST APIs.
- The system shall use a modular monolithic architecture for the MVP.
- Financial operations shall use database transactions.
- Monetary amounts shall use decimal-safe types.

8.2 Project Constraints

- The project shall be developed as a portfolio project.
- The MVP shall not process real money.
- The project shall be developed within the planned ten-week schedule.
- Development resources shall initially consist of one primary developer.
- The Android application shall be the initial client interface.

8.3 Regulatory and Operational Limitations

Nevyl shall be a simulated system and shall not be presented as a licensed financial institution.

The system shall not be used to hold customer deposits, process real financial transactions, or represent itself as a production banking service.

9. ASSUMPTIONS AND DEPENDENCIES

9.1 Assumptions

- Users shall have compatible Android devices.
- Users shall have network connectivity when accessing the backend.
- All financial activity shall be simulated.
- Staff accounts shall be provisioned through authorized administrative processes.
- Supported currencies shall be configurable.
- The initial project shall use a single backend deployment.
- The database shall be available during normal application operation.

9.2 Dependencies

The project depends on:

- Java Development Kit 21.
- Spring Boot.
- PostgreSQL.
- Android SDK.
- Kotlin.
- Jetpack Compose.
- Gradle and Maven.
- Docker.
- GitHub.
- OpenAPI/Swagger tooling.
- Automated testing frameworks.

10. USE CASES

10.1 UC-001: Customer Registration

Primary Actor: Customer

Preconditions:

- The customer is not already registered.
- The registration service is available.

Main Flow:

1. The customer opens the registration screen.
2. The customer enters required personal information and credentials.
3. The system validates the submitted information.
4. The system checks for duplicate identity fields.
5. The system securely stores the customer record.
6. The system confirms successful registration.

Alternative Flow:

- If the submitted data is invalid, the system returns validation errors.
- If the identity fields already exist, the system rejects the registration.

Postcondition:

- A customer record is created.

10.2 UC-002: Customer Login

Primary Actor: Customer

Preconditions:

- The customer has a registered account.
- The account is active.

Main Flow:

1. The customer submits login credentials.
2. The system validates the credentials.
3. The system issues authentication tokens.
4. The customer gains access to permitted application features.

Alternative Flow:

- Invalid credentials result in an authentication error.

- Suspended or disabled users are denied access.

Postcondition:

- The customer is authenticated.

10.3 UC-003: Internal Funds Transfer

Primary Actor: Customer

Preconditions:

- The customer is authenticated.
- The source account is active and owned by the customer.
- The destination account is eligible.
- Both accounts use the same currency.

Main Flow:

1. The customer selects the source account.
2. The customer enters the destination account number.
3. The customer enters the transfer amount.
4. The system validates account eligibility and ownership.
5. The system checks available funds.
6. The system creates a ledger transaction.
7. The system records balanced debit and credit entries.
8. The system commits the transaction atomically.
9. The system returns a transaction reference.
10. The customer views the transaction confirmation.

Alternative Flow:

- Insufficient funds: the transfer is rejected.
- Invalid destination: the transfer is rejected.
- Currency mismatch: the transfer is rejected.
- Duplicate request: the previous result is returned.
- Processing failure: the transaction is rolled back.

Postcondition:

- The transfer is either fully posted or not posted at all.

10.4 UC-004: Simulated Deposit

Primary Actor: Customer or Teller

Preconditions:

- The actor is authenticated and authorized.
- The destination account is eligible.

Main Flow:

1. The actor selects the target account.
2. The actor enters the deposit amount.
3. The system validates the amount and currency.
4. The system creates a balanced ledger transaction.
5. The system posts the transaction.
6. The system returns a transaction reference.

Postcondition:

- The account balance is updated consistently with the ledger.

10.5 UC-005: Transaction Reversal

Primary Actor: Manager

Preconditions:

- The manager is authenticated.
- The manager has reversal permission.
- The original transaction is eligible for reversal.

Main Flow:

1. The manager selects the original transaction.
2. The manager provides a reversal reason.
3. The system validates reversal eligibility.
4. The system creates compensating ledger entries.
5. The system posts the reversal.
6. The system records the audit event.
7. The system returns the reversal reference.

Postcondition:

- The original transaction remains preserved.
- The compensating transaction is recorded.

11. ACCEPTANCE CRITERIA

The MVP shall be accepted when the following conditions are satisfied.

11.1 Authentication

- Customers can register and log in.
- Invalid credentials are rejected.
- Passwords are not stored in plaintext.
- Protected resources require authentication.

11.2 Account Management

- Customers can create eligible bank accounts.
- Account numbers are unique.
- Account currencies are correctly assigned.
- Account ownership restrictions are enforced.

11.3 Financial Transactions

- Deposits are correctly posted.
- Withdrawals reject insufficient funds.
- Transfers correctly debit and credit the involved accounts.
- Ledger entries balance.
- Financial operations are atomic.
- Duplicate requests are prevented.
- Concurrent operations do not result in incorrect balances.

11.4 Authorization

- Customers cannot access another customer's private account information.
- Tellers cannot perform unauthorized administrative operations.
- Managers can perform permitted supervisory operations.
- Administrators can manage authorized system configuration.

11.5 Audit and Reversals

- Financial events are recorded.
- Reversals create compensating entries.
- Original transaction records remain preserved.
- Unauthorized reversal attempts are rejected.

11.6 Testing and Deployment

- Critical business logic is covered by automated tests.
- Integration tests validate database-backed functionality.
- The backend can run using documented setup instructions.
- Database migrations execute successfully.
- API documentation is available.

12. REQUIREMENTS TRACEABILITY

The following table links high-level objectives to their associated requirements.

Objective	Requirement Groups
Secure customer access	FR-AUTH, FR-RBAC, NFR-SEC
Customer account management	FR-CUS, FR-ACC
Financial operations	FR-DEP, FR-WDR, FR-TRF
Financial integrity	FR-LED, FR-IDM, FR-CON, NFR-FIN
Transaction history	FR-HIS
Financial correction	FR-REV
Operational accountability	FR-AUD
System quality	NFR-PER, NFR-REL, NFR-MNT, NFR-TST
Data integrity	Section 6, Section 7

Detailed requirement to test traceability shall be maintained during the testing phase.

13. GLOSSARY

Term	Description
Available Balance	Funds available for permitted transactions
Account Freeze	Restriction preventing specified financial operations
Debit	A ledger entry that decreases a customer liability balance under Nevyl's accounting convention
Credit	A ledger entry that increases a customer liability balance under Nevyl's accounting convention
Double-Entry Ledger	Accounting approach where every transaction has balanced debit and credit entries
Idempotency Key	A unique request identifier used to prevent duplicate transaction execution
Internal Funding Account	Bank-controlled ledger account representing simulated funding
Ledger Transaction	A financial event containing its associated ledger entries
Posted Transaction	A successfully committed financial transaction
Reversal	A compensating transaction that counteracts an original transaction
Role	A collection of permissions assigned to a user
Transaction Reference	Unique identifier used to identify a financial transaction

14. APPENDICES

Appendix A: Planned Technology Stack

Component	Technology
Mobile Application	Kotlin
UI Framework	Jetpack Compose
Mobile Architecture	MVVM
Backend Language	Java 21
Backend Framework	Spring Boot
Security	Spring Security and JWT
ORM	Spring Data JPA / Hibernate
Database	PostgreSQL
Database Migration	Flyway
API Documentation	OpenAPI / Swagger
Testing	JUnit, Mockito, Testcontainers
Containerization	Docker
Version Control	Git and GitHub
CI/CD	GitHub Actions

Appendix B: Future Enhancements

Potential future features include:

- M-Pesa sandbox integration.
- Foreign exchange conversion.
- Beneficiary management.
- Scheduled transfers.
- Notifications.
- Web-based staff dashboard.
- Enhanced customer verification.
- Statement export to PDF.

- Transaction limits.
- More advanced approval workflows.

These enhancements shall not be considered part of the initial MVP unless formally added to the requirements baseline.

END OF DOCUMENT

Nevyl Digital Banking — Software Requirements Specification

Prepared by Naftal Bosire